27 FEB 2026

AADK TASK 2

Input, Validation & UI Feedback Plan

MindMatrix

1. Feature Scenario

Feature Name: Expense Splitter with Tip

This feature allows the user to enter the total bill amount, number of people, and choose whether to round up the tip. The app then calculates how much each person should pay.
This is useful because when friends go out to eat, it makes it easy to quickly split the bill fairly without manual calculations.

2. Input & State Variables Listing

Input Component	State Variable	Type	Validation Rule
Bill Amount TextField	billAmountInput	String	Must be a number > 0
Number of People TextField	numPeopleInput	String	Must be a number ≥ 1
Round Up Switch	roundUpSwitch	Boolean	No validation needed (true/false)
Calculate Button	(uses all states)	—	Enabled only if inputs are valid

Validation Rules Explained

Bill Amount

- Cannot be empty
- Must be a valid number
- Must be greater than 0

Number of People

- Cannot be empty
- Must be a valid integer
- Must be at least 1

Round Up Switch

- Just true or false
- No validation required

3. UI Feedback & Feedback States

Validation Feedback Plan

Validation Rule	UI Feedback if Invalid
Bill empty	Show error text: “Bill amount required”
Bill ≤ 0	Show error: “Bill must be greater than 0”
People empty	Show error: “Enter number of people”
People < 1	Show error: “At least 1 person required”

UI Feedback Details

- TextField border turns red when invalid
- Error message shown below the TextField
- “Calculate” button becomes disabled (grey) if any input is invalid
- Button becomes enabled only when all inputs are valid

On Successful Input

- State updates correctly
- Tip is calculated (15%)
- If roundUpSwitch = true → round tip up
- Total amount per person is calculated
- Result is displayed in a Text composable

On Invalid Input

- Error message appears
- Button disabled
- No calculation happens
- Result area stays hidden or shows “—”

4. Wireframe & Flow Sketch

Flow of Interaction

- User enters bill amount
- billAmountInput state updates
- Validation runs
- User enters number of people
- numPeopleInput state updates
- Validation runs
- If both valid → Button enabled
- User presses Calculate
- Calculation happens
- Result displayed

Composables I Would Use

- TextField → for bill amount
- TextField → for number of people
- Switch → for round up
- Button → Calculate
- Text → error messages
- Text → result display
- Modifier.enabled(false) → disable button
- Modifier.border() or isError = true → show red error state

5. Reflection

Validation and user feedback are very important because they prevent wrong data from entering the system. If we don't validate inputs, the app may crash or show incorrect results. In a reactive UI like Compose, wrong state values can quickly cause wrong UI updates.
For example, if the user enters 0 people and we don't validate it, the app might divide by zero and crash. So validation keeps both the UI and state reliable and safe.